

Otimização de Multiplicação de Matrizes com OpenMP: vetorização, granularidade e padrão de acesso à memória

Trabalho Final — Software de Alto Desempenho (INF01094)

João Vitor do Amaral Spolavore

Instituto de Informática — UFRGS

2026/1

Problema e aplicação

- **Multiplicação de matrizes densas** $C = A \times B$, dupla precisão, $N = 1024$.
- $2N^3 \approx 2,15$ GFLOP por execução; matrizes de 8 MiB (24 MiB no total).
- Núcleo de álgebra linear, simulação e aprendizado de máquina; caso clássico de kernel com **reuso de dados** e **paralelismo abundante**.

Pergunta da investigação

Dado um código sequencial com gargalo conhecido, *quais intervenções OpenMP produzem ganho real, quais não produzem, e por quê?*

- O objetivo **não** é competir com BLAS, e sim explicar a relação entre arquitetura, técnica e resultado.

Nó hype5 (alocação exclusiva via SLURM)

- 2× Intel Xeon E5-2650 v3 @ 2,3 GHz (Haswell, AVX2);
- 10 cores por socket → **20 cores físicos / 40 threads HT**;
- **2 nós NUMA**; L3 de 25 MiB por socket;
- 128 GB DDR4; gcc 12.2 (Debian 12).

Execução

- Job não-interativo (sbatch);
- repo clonado no \$SCRATCH do nó;
- compilação no próprio nó:
-O3 -march=native -fopenmp;
- governador DVFS: performance;
- resultados copiados ao \$HOME (NFS).

Some experiments in this work used the PCAD infrastructure, <http://pcad.inf.ufrgs.br>, at INF/UFRGS.

v0: laços i-j-k canônicos

```
for (i = 0; i < n; i++)  
  for (j = 0; j < n; j++) {  
    double sum = 0.0;  
    for (k = 0; k < n; k++)  
      sum += A[i*n+k] * B[k*n+j];  
    C[i*n+j] = sum;  
  }
```

Baseline: 2,298 s ($\sigma = 0,0003$) $\rightarrow$ 0,94 GFLOP/s

Metodologia

- 1 warm-up não medido + **5 repetições**; mediana $\pm$ desvio ($< 1\%$);
- threads fixadas nos cores físicos (OMP_PROC_BIND=close, OMP_PLACES=cores);
- mesmas entradas em todas as versões (gerador determinístico);
- **validação**: comparação elemento a elemento com a v0 — diferença **zero** nas 4 versões.

Diagnóstico do gargalo

- Pico teórico do nó ≈ 736 GFLOP/s ($20 \text{ cores} \times 16 \text{ flops/ciclo} \times 2,3 \text{ GHz}$).
- A v0 atinge **0,94 GFLOP/s** $\approx$ **0,1% do pico do nó** $\rightarrow$ o gargalo **não** é capacidade de cálculo.
- Laço interno lê $B[k*n+j]$ com **stride de N elementos (8 KiB)**: uma linha de cache nova por iteração, usando 1 dos 8 doubles trazidos.
- Sintoma do mapa de decisão da disciplina: *tempo dominado por memória / acessos com stride*.

Hipóteses de intervenção

- | | |
|---|--------------|
| H1. Vetorizar o produto escalar (<code>omp simd</code>) aproveita as unidades SIMD. | (falsa) |
| H2. Paralelizar o laço externo com granularidade grossa escala com os cores. | (verdadeira) |
| H3. Trocar a ordem dos laços elimina o stride — ataca a causa raiz. | (verdadeira) |

Otimização v1 — omp simd no laço interno (sem ganho)

```
double sum = 0.0;
#pragma omp simd reduction(+:sum)
for (k = 0; k < n; k++)
    sum += A[i*n+k] * B[k*n+j];
```

- **Hipótese (H1):** SIMD $\approx$ 4 doubles/instrução (AVX2) $\rightarrow$ ganho esperado $\sim 4\times$.

- O compilador **vetoriza de fato** (-fopt-info-vec), mas monta os vetores de B com cargas escalares — o acesso tem stride.
- O tráfego de memória por iteração **não muda**.

Resultado medido

2,388 s $\rightarrow$ **0,96** $\times$ (leve piora)

v2: laço externo (grossa)

```
#pragma omp parallel for \  
    schedule(static)  
for (i = 0; i < n; i++)  
    ... /* corpo igual ao da v0 */
```

- 1 região paralela; N/p linhas por thread, sem sincronização interna.
- Corpo idêntico à v0 $\rightarrow$ isola o efeito do paralelismo (v2@1 = 2,294 s $\approx$ v0).

v3: laço interno (fina)

```
for (i ...) for (j ...) {  
    #pragma omp parallel for \  
        reduction(+:sum)  
    for (k = 0; k < n; k++) ...  
}
```

- Região paralela criada $N^2 \approx 10^6$ vezes;
- cada uma distribui só 1024 multiplicações e termina em reduction + **barreira entre 2 sockets**.

Otimização v4 — troca de laços i-k-j (segunda iteração do ciclo)

```
#pragma omp parallel for \  
    schedule(static)  
for (i = 0; i < n; i++) {  
    for (j ...) C[i*n+j] = 0.0;  
    for (k = 0; k < n; k++) {  
        const double a = A[i*n+k];  
        for (j = 0; j < n; j++)  
            C[i*n+j] += a * B[k*n+j];  
    }  
}
```

- Após a v2 o ciclo volta ao diagnóstico: throughput ainda $\approx 2\%$ do pico $\rightarrow$ o stride continua lá.
- Laço interno passa a percorrer B e C com **stride 1**: vetorização efetiva + prefetch sequencial + linha de cache usada por inteiro.
- Soma na mesma ordem de $k \rightarrow$ resultado **bit a bit idêntico** ao da v0.

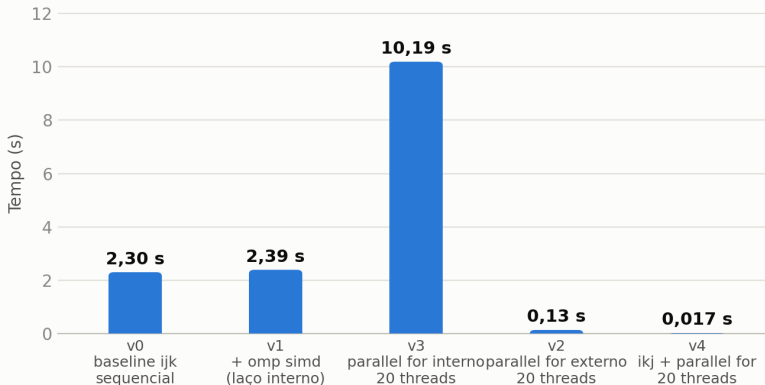
Resultados (hype, $N = 1024$, mediana de 5 repetições)

Versão	Configuração	Tempo	Speedup	GFLOP/s	Efic.
v0	sequencial	2,298 s	1,00×	0,94	—
v1	omp simd, sequencial	2,388 s	0,96 ×	0,90	—
v3	paralelo interno, 20 threads	10,193 s	0,23 ×	0,21	1%
v2	paralelo externo, 20 threads	0,132 s	17,47×	16,3	87%
v4	i-k-j + paralelo, 20 threads	0,017 s	132 ×	123,5	83%

- Validação: diferença máxima **zero** vs. v0 nas 4 versões.
- Eficiência paralela = $\text{speedup}(p)/p$, relativa à mesma versão com 1 thread.
- Uma otimização **piorou 4,4×** — discutida, não escondida.

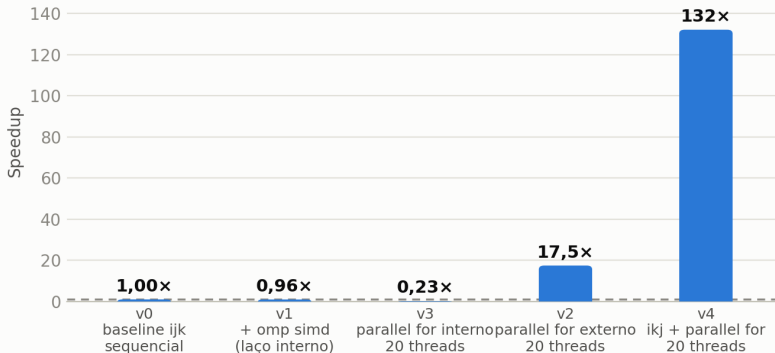
Tempo de execução por versão

Matmul N=1024, dupla precisão — mediana de 5 repetições — menor é melhor



Speedup por versão (vs. baseline v0)

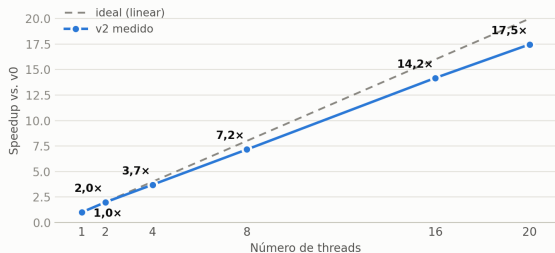
Matmul N=1024 — maior é melhor; linha tracejada marca o baseline (1x)



Escalabilidade da v2: o gráfico que explica o resultado

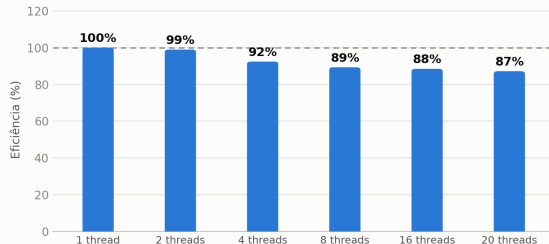
Escalabilidade da v2: speedup vs. número de threads

Matmul N=1024 — pontos acima da linha ideal indicam speedup superlinear (ver discussão)



Eficiência paralela da v2

$\text{eficiência}(p) = \text{speedup}(p) / p$, relativa à v2 com 1 thread — acima de 100% = superlinear



Escalonamento quase linear até 20 threads (99% → 87%); a queda gradual acompanha a fronteira NUMA (threads > 10 ocupam o segundo socket).

Extra: Hyper-Threading (40 threads, OMP_PLACES=threads)

Versão	20 threads (cores)	40 threads (HT)	Ganho do HT
v2	0,132 s	0,126 s	+4%
v4	0,0174 s	0,0126 s	+38%

- Na v2 (latência de memória por acesso strided) o segundo thread por core quase não encontra recursos ociosos para usar.
- Na v4 (vetorizada, limitada por banda/latência com acesso sequencial) o HT **esconde latência** e rende +38%.
- Mesma técnica de hardware, ganho dependente do gargalo do código — coerente com a tese do trabalho.

Discussão: por que cada resultado aconteceu

- **v1 (0,96×)**: SIMD multiplica cômputo, mas o tempo é gasto esperando memória; vetorizar sem mudar o padrão de acesso não altera o tráfego. *Uma técnica só é boa otimização quando resolve o gargalo dominante.*
- **v3 (0,23×)**: 10^6 regiões paralelas $\Rightarrow$ overhead de fork/join + reduction + barreiras **atravessando 2 sockets NUMA** custa mais que todo o trabalho útil. Granularidade fina destrói o ganho do paralelismo.
- **v2 (17,5×, 87%)**: granularidade grossa escala quase linearmente; a queda suave de eficiência coincide com a ocupação do segundo socket (acessos remotos NUMA).
- **v4 (132×)**: o par v1/v4 fecha a prova — as mesmas unidades SIMD rendem 0,96× com stride e $7,9\times$ sequencial (1 thread) quando o acesso é corrigido; com 20 threads, 123,5 GFLOP/s ($\approx 17\%$ do pico).

- Sem contadores de hardware (`perf`) no ambiente: o diagnóstico usa métricas derivadas (GFLOP/s, eficiência, escalabilidade) e análise do padrão de acesso.
- Matrizes inicializadas pela thread 0 (first-touch no socket 0): com > 10 threads parte dos acessos é remota; afeta todas as versões igualmente, mas limita o teto de escalabilidade.
- A v4 usa $\approx 17\%$ do pico do nó: os próximos passos seriam tiling/blocking para registradores e L1, e N maiores que estourem a L3.
- Tempos da v4 com 20 threads (≈ 17 ms) são curtos; N maior daria mais folga entre sinal e ruído.

Melhor configuração

v4 — troca de laços i-k-j + `omp parallel for schedule(static)`, 20 threads fixadas nos cores físicos: **0,017 s**, **speedup de 132×** sobre o baseline, com resultado bit a bit idêntico.

Principal aprendizado técnico

O valor de uma técnica depende do gargalo, não da técnica. Sobre o mesmo kernel, construções OpenMP produziram 0,96× (SIMD sem mudar o tráfego), 0,23× (paralelismo com granularidade fina em 2 sockets), 17,5× (granularidade grossa) e 132× (paralelismo + correção do padrão de acesso apontado pelo diagnóstico).

- Medição controlada (warm-up, mediana, pinning, governor, validação) foi o que permitiu distinguir e explicar os quatro desfechos.

- Código, job SLURM, dados brutos e gráficos: github.com/Spolavore/hpc-final

```
# no front-end do PCAD
```

```
curl -O https://raw.githubusercontent.com/Spolavore/\
      hpc-final/main/bench/pcad/hype.slurm
```

```
sbatch hype.slurm      # resultados em ~/resultados-hpc-final/<jobid>/
```

```
# na maquina local
```

```
python3 analysis/plot.py results-hype/raw.csv
```

Some experiments in this work used the PCAD infrastructure, <http://pcad.inf.ufrgs.br>, at INF/UFRGS.